Powerful Linux Commands and Basic shell scripting for VLSI Design Tasks

Mastering grep, sed, awk & basic shell scripting

Sampath Voonna

VLSI Expert, Mysuru

Introduction

- Commands like **grep**, **sed**, and **awk** help designers quickly search, filter, and process text-based reports generated by EDA tools.
- This reference reflects my hands-on learning of Linux power tools during my VLSI training and focuses on practical command-line workflows used in real-world EDA environments.
- These commands and scripts help automate repetitive tasks, improve debugging efficiency, and speed up analysis of synthesis, timing, and verification reports.
- I have learned these commands during my VLSI Physical Design training at VLSI Experts Pvt. Ltd, Mysuru.

GitHub: https://github.com/VoonnaSampath
LinkedIn: https://www.linkedin.com/in/gurunadh-sampath-voonna-154939223/

grep – Pattern Searching Power Tool

Command	Description / VLSI Use Cases
`grep "Error" file.log`	To Search for the pattern (Error) in the synthesis or simulation log files
`grep -i "warning" *.rpt`	Case-insensitive search to capture all the warnings in reports
`grep -r "module" .`	To recursive search and find the pattern (module) in the current directory
`grep -n -C 3 "clk" design.v`	Shows 3 lines before and after a match along with line numbers
`grep -w "reset" *.v`	To match whole word only in all the verilog files
`grep -c "Error" *.log`	Counts total matching lines with pattern (Error) per log file
`grep -v "INFO" run.log`	To exclude matching lines to hide non-critical messages in the log file
`grep -E "Error|Warning" timing.rpt`	Extended regex search to extract key timing metrics from the report
`grep -l "Error" *.log`	Lists only the file names that contain the pattern (Error)
`grep -H "setup" *.rpt`	Displays file name along with matching pattern (setup) lines
`grep -A 5 "WNS" timing.rpt`	Shows 5 lines after the match to analyze timing context
`grep "^#" script.tcl`	Searches lines starting with # (comments) in TCL script file

sed – Stream Editor for Automation

Command	Description / VLSI Use Cases
`sed 's/old/new/' file.v`	Replaces the first occurrence of a signal or module name (old) with (new) in each line of the Verilog file
`sed 's/old/new/g' file.v`	Replaces all occurrences of a signal, module, or parameter name (old) with (new) across the entire Verilog file
`sed -n '10,20p' report.rpt`	Prints lines 10 to 20 from a synthesis or timing report to inspect a specific analysis section
`sed '/^$/d' file.txt`	Deletes all empty lines from synthesis, timing, or power reports for clean formatting
`sed '1d' file.log`	Removes the first line (tool header or version info) from the EDA log file
`sed '$d' file.log`	Removes the last line (summary or statistics footer) from the log file (synthesis or timing)
`sed 's/#.*//' script.tcl`	Removes comment lines from TCL constraint or run scripts to view active commands only
`sed -i 's/clk/clk_i/g' design.v`	Renames a clock signal (clk) to an internal clock (clk_i) throughout the Verilog design file
`sed '/ERROR/d' run.log`	Deletes all ERROR messages from simulation or synthesis logs file to isolate warnings or INFO messages
`sed -n '/WNS/p' timing.rpt`	Prints only Worst Negative Slack (WNS) related lines from the timing report
`sed 's/[[:space:]]\+/ /g' report.txt`	Normalizes spacing in timing, power, or area reports for easier parsing and readability

awk – Report Analysis Tool

Command	Description / VLSI Use Case
`awk '{print $1}' file.txt`	Prints the first column such as cell name, net name, or timing path identifier from a report file
`awk '{print $1, $3}' report.txt`	Prints the selected columns like timing path name and slack value from the report
`awk '{sum+=$2} END {print sum}' data.txt`	Calculates the total area or power by summing a column in the report
`awk '$3 < 0 {print $0}' timing.rpt`	Prints only timing paths with negative slack to identify setup or hold violations
`awk 'END{print NR}' file.txt`	Counts the total number of entries such as timing paths, cells, or nets in the report
`awk '{max=($2>max)?$2:max} END{print max}' data.txt`	Finds the maximum value of the mentioned column such as worst delay, highest power, or largest area entry
`awk '{print NR, $0}' file.txt`	Prints line numbers along with line numbers (report entries) for easier debugging and traceability
`awk '{count[$1]++} END {for(i in count) print i, count[i]}' file.txt`	Counts repeated occurrences in the mentioned column such as cell names, modules, or timing paths in report
`awk '$1=="clk" {print $0}' design.rpt`	In the mentioned column prints only clock-related entries such as clock nets or clock paths from report
`awk '{print $1 "\t" $2}' report.txt`	Prints formatted output (e.g., cell name and delay) with tab spacing for clean readability

Beginner-Level Linux Shell Scripts

1. List and View All DC Reports

```
#!/bin/bash
cd dc_reports

echo "Listing all DC report files:"
ls *.txt

echo "Showing first 10 lines of timing report:"
head timing.txt
```

2. Find Errors and Warnings from All Reports

```
#!/bin/bash
cd dc_reports

echo "Searching for ERROR messages:"
grep -i "error" *.txt

echo "Searching for WARNING messages:"
grep -i "warning" *.txt
```

3. Extract Only Timing Violations

```
#!/bin/bash
cd dc_reports

echo "Extracting negative slack paths..."
grep "slack" timing.txt | awk '$NF < 0 {print $0}' > neg_slack.txt

echo "Negative slack report created"
wc -l neg_slack.txt
```

4. Clean DC Reports by Removing Extra Spaces

```
#!/bin/bash
cd dc_reports

sed 's/[[:space:]]\+/ /g' area.txt > area_clean.txt
echo "Cleaned area report generated"
```

5. Find All Timing Reports in the Project

```
#!/bin/bash

echo "Finding all timing-related reports:"
find . -name "*timing*.txt"
```